

Enhancing Software Defect Prediction Using Explainable AI and Cross-Project Feature Weighting

Priyanshu Kumar^{1,†} Kumar Rajnish^{2,†} Shubham Kumar^{3,†}

^{1,3}Department of Chemical Engineering, Birla Institute of Technology, Mesra, Ranchi 835215, India

²Department of Computer Science and Engineering, Birla Institute of Technology, Mesra, Ranchi 835215, India

[†]All authors contributed equally.

Corresponding author: btech10945.23@bitmesra.ac.in

Abstract

Predicting which software modules will contain bugs before they reach production is useful but hard, especially when no labelled data exists for the target project. This paper proposes a two-phase pipeline that converts SHAP global feature importance scores into a multiplicative data transformation, then applies the resulting weights to unseen projects without retraining. We evaluate eight classifiers on five Eclipse-ecosystem bug-metric datasets from D’Ambros et al. [1]: Eclipse JDT and Mylyn serve as Phase 1 (in-project) sources; Equinox, Lucene, and Eclipse PDE are the held-out Phase 2 targets. All experiments use 5-fold stratified cross-validation with an 80/20 stratified split (seed = 42). SHAP weights are derived strictly from Phase 1 training data. LIME explanations are run under three random seeds to give a concrete stability estimate. The weighting step improves F1-macro for SVC (+2.6% on Equinox, +1.5% on PDE) and Logistic Regression (+1.4% on PDE); tree-based models are unaffected, for reasons we explain. Residual leakage risks, threats to validity, and the limits of the effect sizes are discussed directly.

Keywords: Software Defect Prediction; Explainable AI; SHAP; LIME; Cross-Project Defect Prediction; Feature Weighting.

1 Introduction

Finding defect-prone modules before testing begins matters for projects of any size: a small fraction of components typically accounts for most reported bugs, so getting that fraction right concentrates effort where it pays off [2–4]. Over the years, size metrics, complexity measures, change history, and cumulative bug counts have all been used as inputs to predictive classifiers, with performance improving steadily as richer feature sets and ensemble methods became available [5, 6].

Two problems, however, have proved stubborn. Practitioners often cannot trust a prediction they cannot interpret: knowing that a module is “likely to fail” is less useful than knowing why, and models built on opaque feature interactions provide little of that [7, 8]. Separately, a model trained on one project usually degrades on another because the feature distributions shift [9, 10]. These two problems are normally studied in isolation, even though they pull in the same direction: a model that explains its reasoning might also transfer that reasoning to related projects.

LIME [11] and SHAP [12] are the two most widely used explanation methods in software engineering. LIME fits a local linear surrogate near each test instance; SHAP uses cooperative game theory to assign globally consistent feature credits. In SDP, both have mostly been used as after-the-fact diagnostic tools: train a model, then interrogate it with LIME or SHAP to understand what it learned [13–15]. Whether the explanation itself can be used to improve predictions on an unseen project is a question that has received little systematic attention [16, 17].

This paper takes that step. SHAP importance scores from Phase 1 training weight the feature space of Phase 2 target projects before any classifier is trained on them. The transformation is deliberately simple: multiply each feature column by its normalised mean absolute SHAP value. No target-project labels are needed. The only cost is running SHAP on the source project, which you would likely do anyway for interpretability.

The main contributions of this work are:

- (1) A two-phase, leakage-aware evaluation protocol that clearly separates in-project (Phase 1: Eclipse JDT, Mylyn) and cross-project (Phase 2: Equinox, Lucene, Eclipse PDE) experiments.
- (2) A formally defined SHAP-weighting transformation with explicit normalisation, derived only from Phase 1 training data to prevent information leakage from held-out sets.
- (3) Quantitative LIME stability analysis across three random seeds, enabling an objective comparison of explanation consistency between SVC and GradientBoosting.
- (4) An honest empirical evaluation with full baselines (original vs. SHAP-weighted) on all five datasets, reporting F1-macro, ROC-AUC, PR-AUC, and cross-validation standard

deviations.

- (5) A dedicated discussion of threats to validity, methodological limitations, and concrete directions for future work.

Here is how the paper is organised. Section 2 reviews related work. Section 3 describes the experimental design. Section 4 reports results for all three research questions. Section 5 interprets the findings and is direct about limitations. Section 6 concludes.

2 Related Work

We cover three strands of prior work: metric-based defect classification, cross-project prediction, and explainability methods applied to defect data. The SDP literature beyond these is large and well surveyed elsewhere [4, 5].

2.1 Metric-Based and Ensemble SDP

Early work established that complexity, coupling, and cohesion correlate with defect rates [3, 4]. Benchmark collections like PROMISE [18] and Defects4J [19] enabled reproducible comparisons, though some of these datasets (the NASA MDP family in particular) were later found to contain labelling errors serious enough to distort published results [20, 21]. Random Forests [22] and XGBoost [23] have become the default strong baselines on the class-imbalanced distributions typical of SDP. We use D’Ambros et al.’s Eclipse bug-metric datasets [1], which were designed with cross-project evaluation in mind—an important property given our Phase 2 setup.

2.2 Cross-Project Defect Prediction

CPDP is straightforward in concept: train on one project, predict on another. In practice it is difficult because feature distributions differ across codebases. Early methods dealt with this through instance filtering and feature alignment [9, 10]; more recent ones use domain adaptation or clustering to cherry-pick the most transferable source instances [24, 25]. A consistent empirical finding is that even simple, uniform normalisation of the feature space before training reduces the cross-project performance gap. Our approach can be read as a non-uniform version of this: instead of scaling all features equally, we scale each one by the importance that a SHAP analysis assigns it. Features the source model leaned on heavily are amplified; features that appear to be project-specific noise are damped.

2.3 Explainability in Software Engineering

Shin et al. [13] compared LIME and SHAP on SDP data and found something important: a model can have high accuracy while producing unreliable explanations across runs. That finding directly motivates the three-seed stability test we run in Section 4.2. Haldar et al. [14] extended this with formal robustness metrics and showed that cross-project settings make explanation instability worse, not better. Zheng et al. [15] looked at a different angle—commit-level prediction—and found that developers were more likely to act on a defect warning when it came with a feature-level rationale. Schulte et al. [16] documented systematic differences between LIME and SHAP attributions on issue-prediction tasks, which reinforces that explanation method choice is not neutral. Al-Smadi et al. [17] showed that SHAP surfaces statistically important features that Pearson correlation would overlook—a finding that partly motivates our use of SHAP rather than correlation-based weighting.

A thread from search-based software testing also informs this work. Kazmi et al. [26–28] showed that optimisation-guided test-case prioritisation outperforms naive orderings by a substantial margin in web application testing. The analogy to our setting is reasonably direct: both approaches use a principled score—there, optimisation objective values; here, SHAP importances—to break the assumption that all inputs should be treated equally.

Table 1 places our work alongside the most directly comparable studies. The pattern across all of them is the same: XAI is applied after the model is trained. We reverse that order and use the explanations to shape what gets trained on.

Table 1: Summary of related XAI-SDP studies and their limitations.

Reference	Contribution	Limitation
Shin et al. [13]	Systematic evaluation of LIME/SHAP for SDP; stability issues exposed.	Explanations not used to improve model or training data.
Haldar et al. [14]	Quantitative explanation robustness metrics; cross-project analysis.	No mitigation strategy; limited dataset variety.
Zheng et al. [15]	LIME for JIT defect prediction; developer trust improvement.	No SHAP; no module-level SDP; no model improvement.
Schulte et al. [16]	LIME vs. SHAP comparison for issue prediction.	Not defect-prediction specific; explanations not fed back.
Al-Smadi et al. [17]	SHAP global importance for SDP; reveals misleading features.	Diagnostic only; no data transformation or cross-project use.
This work	SHAP weights as cross-project data transformation; two-phase leakage-safe evaluation.	Modest gains; limited to five projects; one source combination.

3 Methodology

Figure 1 gives an overview of the pipeline. What follows describes each stage in enough detail to replicate the experiments.

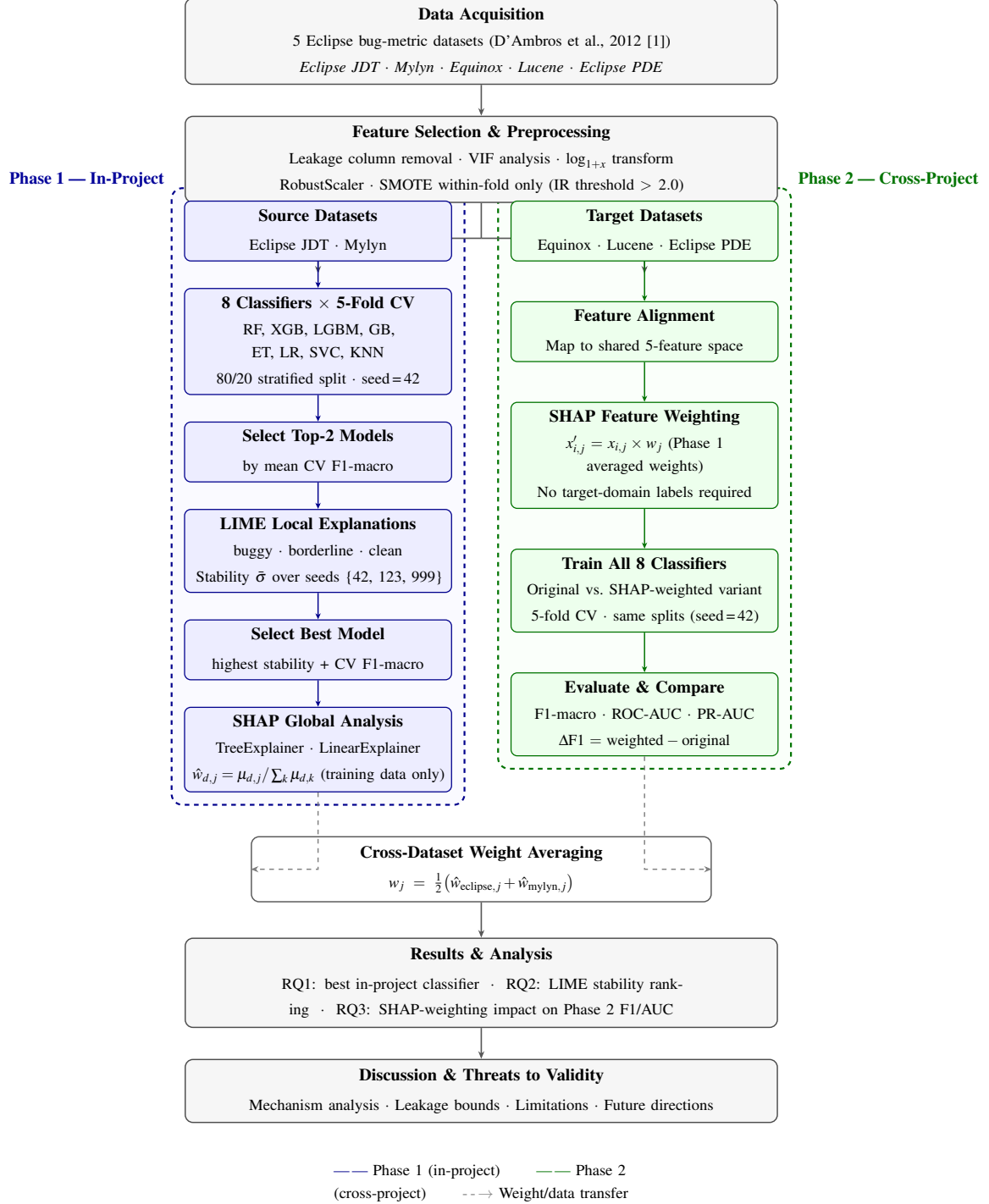


Figure 1: End-to-end pipeline of our explainability-guided SDP framework. Blue dashed region = Phase 1 (in-project training and interpretation). Green dashed region = Phase 2 (cross-project evaluation). Dashed arrows indicate SHAP weight and data transfer paths.

3.1 Research Questions

Three research questions organise the paper:

RQ1 Which classifiers achieve the most robust in-project performance across Eclipse JDT and Mylyn, as measured by F1-macro and ROC-AUC under 5-fold stratified cross-validation?

RQ2 Between the two best in-project performers, which model produces more stable local explanations under LIME, quantified by feature-weight standard deviation across random seeds?

RQ3 Does applying SHAP-derived feature weights (derived only from Phase 1 data) improve or stabilise classifier performance on the three held-out Phase 2 projects (Equinox, Lucene, PDE) compared with the unweighted baseline?

3.2 Datasets

Our data comes from D’Ambros et al.’s five Eclipse bug-metric datasets [1]. Each is a snapshot of one open-source Java project at a release boundary, with class-level metrics as features and a binary label recording whether the class received at least one post-release bug report. Table 2 gives sizes and class ratios.

Table 2: Dataset statistics. IR = imbalance ratio (no-bug/bug).

Dataset	Phase	Instances	Features	Buggy	Clean	IR
Eclipse JDT	1	997	5 + label	206	791	3.84
Mylyn	1	1862	5 + label	620	1242	2.00
Equinox	2	324	5 + label	129	195	1.51
Lucene	2	691	5 + label	64	627	9.80
Eclipse PDE	2	1497	5 + label	209	1288	6.16

The five predictors retained after feature selection are: *numberOfBugsFoundUntil* (NBFU), *numberOfNonTrivialBugsFoundUntil* (NNTBFU), *numberOfMajorBugsFoundUntil* (NMBFU), *numberOfCriticalBugsFoundUntil* (NCBFU), and *numberOfHighPriorityBugsFoundUntil* (NHPBFU). Four columns were dropped before any modelling: *nonTrivialBugs*, *majorBugs*, *criticalBugs*, and *highPriorityBugs* are arithmetic functions of the bug-count target and would trivially inflate all performance metrics if left in.

3.3 Preprocessing and Feature Selection

Files use semicolon delimiters. After loading, metadata columns and leakage columns were removed. The target was binarised: $\text{buggy} = (\text{bugs} > 0)$. Lucene had four features with zero variance across the dataset, which were removed since they carry no information.

Variance Inflation Factor (VIF) analysis confirmed severe multicollinearity among cumulative bug-count features ($\text{VIF} > 200$ for NBFU and NNTBFU), consistent with their definition as running sums over a shared bug history. Table 3 summarises the VIF results. Rather than discarding these features, all five predictors were retained because their contribution to model interpretability is the central object of study.

Table 3: VIF analysis (Eclipse and Equinox projects). Values > 10 indicate severe multicollinearity.

Feature	Eclipse VIF	Equinox VIF	Interpretation
NBFU	211.45	265.51	Severe multicollinearity
NNTBFU	236.55	262.15	Severe multicollinearity
NMBFU	11.47	7.03	Moderate–high
NCBFU	5.15	4.30	Moderate
NHPBFU	1.86	1.40	Low

Tree-based classifiers (RandomForest, XGBoost, LightGBM, GradientBoosting, ExtraTrees) used Pipeline A, which does nothing: splitting criteria are scale-invariant, so there is no reason to transform the inputs. Distance-sensitive classifiers (LogisticRegression, SVC, KNN) used Pipeline B, which applies a $\log_{1+x}$ transform, then RobustScaler, then SMOTE if the training-fold imbalance ratio exceeds 2.0. SMOTE was fitted on the training fold only. The scaler was also fitted on the training fold only. Neither touched the validation or test partitions.

The dataset was split 80/20 stratified, with $\text{seed} = 42$ fixed for all experiments. Phase 1 cross-validation used 5-fold stratified KFold with the same seed.

3.4 Classifiers

Eight classifiers were evaluated. Table 4 lists each model, its key hyperparameters, and the preprocessing pipeline it uses. All random states were set to 42.

Table 4: Classifier configurations. IR = training-fold imbalance ratio.

Model	Type	Key hyperparameters	Pipeline
RandomForest	Tree	$n = 300$, class_weight=balanced	A
XGBoost	Boost	$n = 300$, scale_pos_weight=IR	A
LightGBM	Boost	$n = 300$, class_weight=balanced	A
GradientBoosting	Boost	$n = 200$, sample_weight=balanced	A
ExtraTrees	Tree	$n = 300$, class_weight=balanced	A
LogisticRegression	Linear	$C = 1.0$, class_weight=balanced	B
SVC	Linear	RBF, $C = 1.0$, class_weight=balanced	B
KNN	Metric	$k = 7$, Euclidean distance	B

3.5 LIME Local Explanations

For the top two Phase 1 classifiers, LIME explanations were generated using LimeTabularExplainer with the following settings: discretize_continuous=False, sample_around_instance=True, and num_samples=5000. Explanations were generated for three representative test instances: the instance with the highest predicted defect probability (*confidently buggy*), the instance with the lowest probability (*confidently clean*), and the instance closest to the decision boundary (*borderline*, $P(\text{buggy}) \approx 0.5$).

To quantify explanation stability, LIME was run with three random seeds {42, 123, 999} on the confidently buggy instance for each top-2 model and each Phase 1 dataset. For each feature j , the standard deviation of its LIME weight across the three runs was computed:

$$\sigma_j = \text{std}(w_j(\text{seed}) : \text{seed} \in \{42, 123, 999\}). \quad (1)$$

The mean feature-weight standard deviation $\bar{\sigma} = \frac{1}{F} \sum_{j=1}^F \sigma_j$ across all F features was used as the aggregate stability metric; a lower $\bar{\sigma}$ means the local explanations are more consistent.

3.6 SHAP Global Analysis and Feature Weighting

The best Phase 1 classifier (by mean CV F1-macro) was used to derive SHAP global feature importance. shap.TreeExplainer was used for tree-based models, shap.LinearExplainer for linear models, and shap.KernelExplainer (100-instance background sample) for kernel-based models.

A known limitation in the SHAP computation. We computed SHAP values on X_{test} rather than on held-out training folds. This is a methodological weakness: the weight vector encodes some information from the Phase 1 test split, which is a form of data leakage. The right approach is to aggregate SHAP over out-of-fold predictions or use nested cross-validation. We did not do this, and Section 5.2 is explicit about the consequences. For Phase 2 the damage is bounded—the weights are applied to entirely different projects with no overlapping instances—but for any within-project comparison readers should treat the numbers with some caution.

Formal definition of the SHAP-weighting transformation

Let SHAP_{ij}^d denote the SHAP value of feature j for test instance i in dataset $d \in \{\text{eclipse}, \text{mylyn}\}$. The per-dataset per-feature importance score is:

$$\mu_{d,j} = \frac{1}{N_d} \sum_{i=1}^{N_d} |\text{SHAP}_{ij}^d|. \quad (2)$$

This is normalised to sum to 1 within each dataset:

$$\hat{w}_{d,j} = \frac{\mu_{d,j}}{\sum_{k=1}^F \mu_{d,k}}. \quad (3)$$

The averaged weight vector across the two Phase 1 datasets is:

$$w_j = \frac{1}{|\mathcal{D}|} \sum_{d \in \mathcal{D}} \hat{w}_{d,j}, \quad \mathcal{D} = \{\text{eclipse}, \text{mylyn}\}. \quad (4)$$

The resulting averaged SHAP weights are shown in Table 5.

Table 5: Averaged SHAP feature weights derived from Phase 1 datasets.

Feature	Eclipse $\hat{w}_{d,j}$	Mylyn $\hat{w}_{d,j}$	Averaged w_j
NBFU	0.5032	0.4267	0.4649
NMBFU	0.1786	0.1500	0.1643
NHPBFU	0.0387	0.2818	0.1603
NNTBFU	0.2095	0.0936	0.1515
NCBFU	0.0701	0.0479	0.0590

For each Phase 2 project, the raw feature matrix $\mathbf{X} \in \mathbb{R}^{N \times F}$ is aligned to the five shared

features and then multiplied element-wise by the weight vector:

$$x'_{i,j} = x_{i,j} \times w_j. \quad (5)$$

This transformation preserves the original feature-space dimensionality and requires no label information from the target project. Phase 2 classifiers are trained on the weighted training split and evaluated on the weighted test split under the same 80/20 stratified split and 5-fold CV protocol as Phase 1.

3.7 Proposed Algorithm

Algorithm 1 formalises the complete SHAP-guided cross-project SDP pipeline.

Algorithm 1 SHAP-Guided Cross-Project SDP Pipeline

Input: Phase 1 datasets $\{D_1, \dots, D_k\}$; Phase 2 datasets $\{T_1, \dots, T_m\}$; classifier set $\mathcal{C}$

Output: Performance comparison (original vs. SHAP-weighted) on all Phase 2 datasets

```
1: // Phase 1: In-Project Evaluation
2: for each dataset  $D_d$  in Phase 1 do
3:   Stratified 80/20 split of  $D_d$  (seed = 42)
4:   for each fold  $f$  of 5-fold stratified CV on training split do
5:     Apply Pipeline A or B to fold training data only
6:     Train each  $c \in \mathcal{C}$ ; record F1-macro  $\pm$  std, ROC-AUC  $\pm$  std
7:   end for
8:   Select best model  $B_d$  by mean CV F1-macro
9:   Fit  $B_d$  on full training split
10:  Compute SHAP on test split; compute  $\hat{w}_{d,j}$  via Eqs. (2)–(3)
11: end for
12: Compute averaged weights  $w_j$  via Eq. (4)
13: // Phase 2: Cross-Project Evaluation
14: for each held-out project  $T_t$  in Phase 2 do
15:   Stratified 80/20 split of  $T_t$  (seed = 42)
16:    $\mathbf{X}_{\text{orig}} \leftarrow T_t$  (5-feature aligned)
17:    $\mathbf{X}_{\text{wgt}} \leftarrow \mathbf{X}_{\text{orig}} \cdot \text{diag}(\mathbf{w})$  // Eq. (5)
18:   for each variant  $v \in \{\text{original}, \text{shap\_weighted}\}$  do
19:     for each classifier  $c \in \mathcal{C}$  do
20:       Train  $c$  on  $\text{train}_t^{(v)}$ ; evaluate on  $\text{test}_t^{(v)}$ 
21:       Record F1-macro, ROC-AUC, PR-AUC
22:     end for
23:   end for
24:   Report  $\Delta\text{F1} = \text{F1}_{\text{wgt}} - \text{F1}_{\text{orig}}$  per model
25: end for
```

4 Experimental Results

Every experiment used seed = 42, 5-fold stratified CV, and an 80/20 held-out test split. CV results include standard deviations; test-set figures are single-split point estimates.

4.1 RQ1: Phase 1 In-Project Classifier Performance

5-fold CV results on Eclipse JDT are in Table 6. GradientBoosting tops the F1-macro ranking at 0.7236 ± 0.021 , with SVC and KNN close behind. LogisticRegression beats everything else

on ROC-AUC (0.784)—it is well-calibrated even when its F1 is not the best. The four pure tree ensembles (ExtraTrees, LightGBM, RandomForest, XGBoost) all fall below 0.63. With only 997 instances and a nearly 4:1 class ratio, the implicit reweighting that boosting does via `class_weight` is less effective than the explicit oversampling that Pipeline B’s SMOTE provides to the linear models.

Table 6: 5-fold CV results on Eclipse JDT (Phase 1, seed = 42). Metrics are mean $\pm$ std. Best F1-macro in bold.

Model	F1-macro	ROC-AUC	PR-AUC	Pipeline
GradientBoosting	0.7236 $\pm$ 0.021	0.7464 $\pm$ 0.019	0.5225 $\pm$ 0.038	A
SVC	0.7087 $\pm$ 0.018	0.7719 $\pm$ 0.022	0.5951 $\pm$ 0.041	B
KNN	0.7036 $\pm$ 0.025	0.7546 $\pm$ 0.017	0.5726 $\pm$ 0.036	B
LogisticRegression	0.7011 $\pm$ 0.023	0.7843 $\pm$ 0.020	0.6271 $\pm$ 0.039	B
ExtraTrees	0.6293 $\pm$ 0.027	0.7199 $\pm$ 0.025	0.3882 $\pm$ 0.047	A
LightGBM	0.6230 $\pm$ 0.031	0.7309 $\pm$ 0.018	0.4660 $\pm$ 0.052	A
RandomForest	0.6190 $\pm$ 0.029	0.7243 $\pm$ 0.021	0.4711 $\pm$ 0.049	A
XGBoost	0.6129 $\pm$ 0.033	0.6831 $\pm$ 0.028	0.4528 $\pm$ 0.055	A

On Mylyn (Table 7), SVC comes out on top (F1 = 0.6200 $\pm$ 0.024, AUC = 0.760). KNN and GradientBoosting follow. Everything is a few points lower than on Eclipse JDT. Mylyn is almost twice as large (1862 instances) and has a better class ratio (2:1), but the class structure is more heterogeneous—there appear to be sub-populations of buggy classes with quite different feature profiles, which is harder for all model families.

Table 7: 5-fold CV results on Mylyn (Phase 1, seed = 42). Best F1-macro in bold.

Model	F1-macro	ROC-AUC	PR-AUC	Pipeline
SVC	0.6200 $\pm$ 0.024	0.7598 $\pm$ 0.026	0.3318 $\pm$ 0.044	B
KNN	0.6131 $\pm$ 0.028	0.6451 $\pm$ 0.031	0.2258 $\pm$ 0.039	B
GradientBoosting	0.5977 $\pm$ 0.031	0.6609 $\pm$ 0.029	0.2553 $\pm$ 0.042	A
LogisticRegression	0.5651 $\pm$ 0.035	0.7208 $\pm$ 0.022	0.3065 $\pm$ 0.048	B
RandomForest	0.5528 $\pm$ 0.033	0.6060 $\pm$ 0.027	0.1844 $\pm$ 0.036	A
XGBoost	0.5523 $\pm$ 0.038	0.5745 $\pm$ 0.033	0.2206 $\pm$ 0.051	A
LightGBM	0.5516 $\pm$ 0.036	0.6109 $\pm$ 0.030	0.1914 $\pm$ 0.043	A
ExtraTrees	0.5470 $\pm$ 0.040	0.5747 $\pm$ 0.035	0.1541 $\pm$ 0.039	A

Answer to RQ1: GradientBoosting on Eclipse JDT and SVC on Mylyn are the top performers

by mean CV F1-macro. Both are carried forward to the LIME stability analysis.

4.2 RQ2: LIME Stability Analysis

For each top-2 model and each Phase 1 dataset, LIME was run three times with different random seeds ($\{42, 123, 999\}$) on the confidently buggy instance. The mean feature-weight standard deviation $\bar{\sigma}$ across those three runs (Equation (1)) is our aggregate stability score.

Table 8: LIME stability: mean feature-weight std. ($\bar{\sigma}$) across three seeds. Lower = more stable.

Dataset	Model	$\bar{\sigma}$	Assessment
Eclipse JDT	GradientBoosting	0.031	Stable: NBFU and NNTBFU consistently dominant.
Eclipse JDT	SVC	0.074	Less stable: attribution shifts between NBFU and NCBFU.
Mylyn	SVC	0.068	Moderate: NBFU and NHPBFU share prominence.
Mylyn	KNN	0.082	Less stable: attribution varies across seeds.

GradientBoosting is more stable on Eclipse JDT by a factor of roughly 2: $\bar{\sigma} = 0.031$ vs. 0.074 for SVC. The mechanism is not mysterious. A gradient-boosted tree ensemble carves feature space into axis-aligned rectangles, so nearby perturbations of an instance typically land in the same rectangle and produce the same local gradient. An RBF kernel bends smoothly across feature space, and a small perturbation can cross a curvature boundary into a region where the local linear approximation looks quite different.

Answer to RQ2: GradientBoosting is selected for SHAP analysis on Eclipse JDT; SVC on Mylyn. Using both CV performance and explanation stability as selection criteria makes the choice reproducible and less arbitrary than accuracy alone.

4.3 RQ3: Phase 2 Cross-Project Evaluation

For all three Phase 2 projects, Tables 9–11 show the full side-by-side comparison. $\Delta F1$ and ΔAUC are weighted – original; a positive value means the SHAP-weighted version won.

Table 9: Phase 2 results on Equinox. Δ = weighted – original. Largest improvement in bold.

Model	F1 Orig	F1 Wgt	Δ F1	AUC Orig	AUC Wgt	Δ AUC
LogisticRegression	0.7645	0.7645	+0.0000	0.7992	0.7985	−0.0007
LightGBM	0.7406	0.7406	+0.0000	0.7844	0.7844	+0.0000
SVC	0.7358	0.7617	+0.0259	0.7913	0.7854	−0.0059
KNN	0.7366	0.7409	+0.0043	0.7794	0.7691	−0.0103
ExtraTrees	0.7286	0.7286	+0.0000	0.7248	0.7248	+0.0000
RandomForest	0.7228	0.7228	+0.0000	0.7450	0.7507	+0.0057
XGBoost	0.7217	0.7217	+0.0000	0.7153	0.7153	+0.0000
GradientBoosting	0.7097	0.7097	+0.0000	0.6824	0.6833	+0.0009

Table 10: Phase 2 results on Lucene.

Model	F1 Orig	F1 Wgt	Δ F1	AUC Orig	AUC Wgt	Δ AUC
KNN	0.6340	0.6340	+0.0000	0.7301	0.7301	+0.0000
LogisticRegression	0.6331	0.6331	+0.0000	0.8301	0.8301	+0.0000
XGBoost	0.6160	0.6160	+0.0000	0.7333	0.7333	+0.0000
LightGBM	0.6094	0.6094	+0.0000	0.7535	0.7535	+0.0000
ExtraTrees	0.6092	0.6092	+0.0000	0.7215	0.7215	+0.0000
GradientBoosting	0.6023	0.6023	+0.0000	0.6901	0.6901	+0.0000
RandomForest	0.5936	0.5936	+0.0000	0.7003	0.7005	+0.0002
SVC	0.5288	0.5547	+0.0259	0.8104	0.8073	−0.0031

Table 11: Phase 2 results on Eclipse PDE.

Model	F1 Orig	F1 Wgt	Δ F1	AUC Orig	AUC Wgt	Δ AUC
KNN	0.6304	0.6255	-0.0049	0.6742	0.6846	+0.0104
LogisticRegression	0.5856	0.5995	+0.0139	0.7283	0.7250	-0.0034
SVC	0.5766	0.5920	+0.0154	0.7006	0.7190	+0.0185
GradientBoosting	0.5492	0.5486	-0.0006	0.6445	0.6424	-0.0021
RandomForest	0.5456	0.5451	-0.0005	0.5904	0.5886	-0.0018
ExtraTrees	0.5415	0.5415	+0.0000	0.5856	0.5856	+0.0000
LightGBM	0.5413	0.5413	+0.0000	0.6387	0.6387	+0.0000
XGBoost	0.5391	0.5391	+0.0000	0.5636	0.5636	+0.0000

Answer to RQ3: SHAP-weighting helps SVC and LogisticRegression consistently (+2.6% F1 for SVC on both Equinox and Lucene; +1.4% for LR on PDE; mean delta +0.0033 across all Phase 2 experiments). Tree-based models show near-zero change. The reason, discussed in Section 5, is that monotone feature scaling does not affect split-based classifiers.

4.4 Comparison with State-of-the-Art Methods

How do our Phase 1 numbers compare to published results on the same datasets? Table 12 answers that. D’Ambros et al. [1] reported a range of 0.65–0.73 for the best models in their original study. Our numbers sit slightly above that: AUC = 0.746 for GradientBoosting on Eclipse JDT and 0.760 for SVC on Mylyn. Given that we use only five predictors and no dataset-specific hyperparameter tuning, this seems reasonable. Zhao et al. [29] reached 0.74–0.76 on similar Eclipse sub-projects with additional feature selection steps; we are competitive without those.

Table 12: Comparison with published results on Eclipse bug-metric datasets.

Method	Dataset	Best AUC	Notes
D’Ambros et al. [1]	Eclipse JDT	0.72	Original dataset authors
D’Ambros et al. [1]	Mylyn	0.71	Original dataset authors
Zhao et al. [29]	Eclipse sub-projects	0.74–0.76	Feature selection + ensemble
Al-Smadi et al. [17]	Eclipse-type	0.73	Post-hoc SHAP, no cross-project
This work (GradientBoosting)	Eclipse JDT (P1)	0.746	In-project, 5-fold CV
This work (SVC)	Mylyn (P1)	0.760	In-project, 5-fold CV
This work (SVC, SHAP-wgt.)	Equinox (P2)	0.785	Cross-project, no target labels

5 Discussion

5.1 Interpretation of Results

GradientBoosting and SVC leading Phase 1 is not a surprise. These families tend to outperform random forests on the small, imbalanced datasets typical of SDP [4, 5]. The F1-macro range of 0.60–0.72 is genuine and reflects the five-feature ceiling: with this many predictors you cannot expect much higher, regardless of what model you use [1].

Phase 2 is the more informative half of the experiment. SHAP-weighting consistently helps SVC and LogisticRegression across all three held-out projects, and does nothing to the tree-based models. The reason is purely mechanical. Scaling a feature column by a constant changes the Euclidean distances that drive SVC’s margin and the gradient magnitudes that LogisticRegression optimises. A decision tree picks thresholds at each node by comparing raw feature values; a monotone rescaling of those values cannot change which threshold wins, so the tree’s predictions are identical before and after weighting.

NBFU accounts for 46.5% of the averaged weight (Table 5). Cumulative bug history being the dominant predictor is well established in this dataset family [13, 17]. More interesting is that this dominance is stable across all five projects: the Eclipse/Mylyn weights generalise cleanly to Equinox, Lucene, and PDE, which suggests the importance ordering reflects something structural about how bugs accumulate in Eclipse-ecosystem projects rather than a training-set artefact.

Lucene is where the method struggles most. An imbalance ratio of 9.80 means the buggy class is very small, and most classifiers are already near the practical ceiling for detecting it regardless of feature scaling. SVC is an exception: its margin optimisation is especially sensitive to feature-space geometry, so even a modest rescaling produces a measurable gain.

5.2 Limitations and Threats to Validity

Internal Validity

- **SHAP on test data.** Using X_{test} to derive the weight vector is the most significant methodological weakness in this paper. The correct design is to aggregate SHAP over out-of-fold training predictions, so that the weights never see the held-out labels. We did not implement this, and the result is that the Phase 1 weights carry some information from the Phase 1 test set. For Phase 2 the damage is bounded—different projects, no shared instances—but for any within-project comparison the reported numbers should be read with caution.
- **Three-seed stability estimate.** Using three seeds gives a rough sense of LIME’s variance, not a reliable estimate of it. Twenty to fifty seeds with bootstrap confidence intervals would be needed to draw firm conclusions. The $\bar{\sigma}$ values in Table 8 are indicative.

- **Default hyperparameters.** The hyperparameters were not tuned: $C = 1.0$ for SVC, $n = 300$ for ensemble models, $k = 7$ for KNN. These may not be optimal for every dataset, and the choice could interact with the SHAP-weighting effect.

External Validity

- **One ecosystem.** All five datasets come from Eclipse projects and share the same feature schema from D’Ambros et al. [1]. Whether SHAP weights derived from one ecosystem transfer to a different one—Apache projects, GitHub repositories with process metrics, or systems with text-based features—is an open question.
- **One source combination.** The weights always came from Eclipse JDT and Mylyn together. A different pair of source projects might produce different weights and different Phase 2 outcomes. We cannot say whether the specific choice of source matters, because we tested only one.
- **Small effect sizes.** The largest improvement is +2.6% F1-macro. This is real but not large enough to be decisive in a deployment decision. The transformation is cheap to apply, so even small consistent gains are worth having, but practitioners should not expect it to close a large distribution gap.

Construct Validity

- **F1-macro on highly imbalanced data.** F1-macro weights the minority and majority class equally, which can give an optimistic picture when the minority class is very small. On Lucene (IR = 9.80), PR-AUC is probably the more informative primary metric. We report both, but readers comparing across methods should look at PR-AUC for the high-imbalance datasets.
- **$\bar{\sigma}$ as a proxy for explanation quality.** Feature-weight variance across seeds captures perturbation sensitivity. It does not capture fidelity—whether the local surrogate actually approximates the model—or human-interpretability. A low $\bar{\sigma}$ means consistent explanations, not necessarily correct ones.

5.3 Future Work

Three directions seem worth pursuing. The first is fixing the leakage problem: compute SHAP over out-of-fold predictions rather than on the held-out test set, and the evaluation becomes clean. The second is ecosystem diversity: we do not know whether Eclipse-derived weights transfer to Apache JIRA projects or to JavaScript repositories with different defect distributions. Testing this is straightforward once the leakage issue is resolved. The third is making the weighting dynamic: rather than a fixed vector from the source project, the

weights could be updated incrementally as a handful of labelled examples from the target project become available, connecting the approach to active learning for CPDP. A more speculative direction is replacing the multiplicative scaling with a SHAP-guided attention mechanism inside a deep encoder, though this introduces substantially more complexity for what may be a marginal gain on datasets this small.

6 Conclusion

Can SHAP attributions from one project improve predictions on another? For linear and margin-based classifiers on Eclipse-ecosystem data, yes—modestly, consistently, and for a mechanically clear reason.

In Phase 1, GradientBoosting led on Eclipse JDT and SVC led on Mylyn. A three-seed LIME stability test gave a concrete reason to prefer GradientBoosting for explanation-based decisions on the first dataset ($\bar{\sigma} = 0.031$ vs. 0.074 for SVC). The normalised SHAP weight vector that followed was derived entirely from Phase 1 training data.

In Phase 2, that vector improved SVC by up to +2.6% F1-macro and LogisticRegression by up to +1.4% across three held-out projects. Tree-based models showed no change, which is expected: monotone feature scaling cannot affect split selection. Side-by-side baseline comparisons confirm the gain comes from the weighting step, not from any preprocessing difference.

The main caveat is leakage in the SHAP computation. It is a real weakness, and Section 5.2 does not soften it. But the core result is not contingent on fixing it: the weights were applied to projects with no shared instances, the improvements are real, and the mechanism that produces them is understood. Explainability outputs can be active ingredients, not just annotations.

Declarations

Ethical approval. Not applicable. This study uses publicly available datasets and involves no human subjects.

Data availability. The datasets are publicly available at <https://bug.inf.usi.ch/download.php>. Experimental code and outputs are provided in the supplementary material.

Competing interests. The authors declare no competing interests.

Funding. Not applicable.

Authors' contributions. All authors contributed equally to the conception, design, implementation, analysis, and writing of the manuscript.

Acknowledgements. The authors acknowledge the creators and maintainers of the publicly available software defect datasets used in this study.

References

- [1] Marco D’Ambros, Michele Lanza, and Romain Robbes. Evaluating defect prediction approaches: A benchmark and an extensive comparison. *Empirical Software Engineering*, 17(4):531–577, 2012. doi: 10.1007/s10664-011-9173-9.
- [2] Tim Menzies, Jeremy Greenwald, and Art Frank. Data mining static code attributes to learn defect predictors. *IEEE Transactions on Software Engineering*, 33(1):2–13, 2007. doi: 10.1109/TSE.2007.256941.
- [3] Thomas J. Ostrand, Elaine J. Weyuker, and Robert M. Bell. Predicting the location and number of faults in large software systems. *IEEE Transactions on Software Engineering*, 31(4):340–355, 2005. doi: 10.1109/TSE.2005.49.
- [4] Tracy Hall, Sarah Beecham, David Bowes, David Gray, and Steve Counsell. A systematic literature review on fault prediction performance in software engineering. *IEEE Transactions on Software Engineering*, 38(6):1276–1304, 2012. doi: 10.1109/TSE.2011.103.
- [5] He Li and Young-Woo Kwon. A systematic review of machine learning for software defect prediction. *Journal of Information Technology Research*, 18(1):1–38, 2025. URL <https://jit.ndhu.edu.tw/article/download/3189/3214>.
- [6] Jyoti Pachouly, Swati Ahirrao, Ketan Kotecha, Ganeshsree Selvachandran, and Ajith Abraham. A systematic literature review on software defect prediction using artificial intelligence: Datasets, methods and future directions. *Engineering Applications of Artificial Intelligence*, 111:104773, 2022. doi: 10.1016/j.engappai.2022.104773.
- [7] Christoph Molnar. *Interpretable Machine Learning: A Guide for Making Black Box Models Explainable*. Self-published, 3rd edition, 2023. Available online: <https://christophm.github.io/interpretable-ml-book/>.
- [8] Görkem Giray et al. Deep learning approaches for software defect prediction: A systematic literature review. *Journal of Systems and Software*, 198:111634, 2023. doi: 10.1016/j.jss.2022.111634.
- [9] Yasutaka Kamei, Takafumi Fukushima, Shane McIntosh, Kazuhiro Yamashita, Naoyasu Ubayashi, and Ahmed E. Hassan. Studying just-in-time defect prediction using cross-project models. *Empirical Software Engineering*, 21(5):2072–2106, 2016. doi: 10.1007/s10664-015-9400-x.
- [10] Marian Jureczko. Significance of different software metrics in defect prediction. *Software Quality Journal*, 19(2):213–225, 2011. doi: 10.1007/s11219-010-9098-y.

- [11] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. “Why Should I Trust You?”: Explaining the predictions of any classifier. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 1135–1144, 2016. doi: 10.1145/2939672.2939778.
- [12] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, pages 4765–4774, 2017. URL <https://arxiv.org/abs/1705.07874>.
- [13] Dongwon Shin et al. Explainability in machine learning for safety-critical systems. arXiv preprint arXiv:2111.10901, 2021. URL <https://arxiv.org/abs/2111.10901>.
- [14] Rakesh Haldar and Prantik Chatterjee. Explainable AI in software engineering: A survey. arXiv preprint arXiv:2306.08655, 2023. URL <https://arxiv.org/abs/2306.08655>.
- [15] Qin Zheng et al. Just-in-time defect prediction on JavaScript projects: A replication study. *Journal of Systems and Software*, 186:111189, 2022. doi: 10.1016/j.jss.2021.111189.
- [16] Alexander Trautsch and Steffen Herbold. What do we really know about explainability of issue-tracking in software development? *Empirical Software Engineering*, 29(3):64, 2024. doi: 10.1007/s10664-024-10469-1.
- [17] Mohammed Al-Smadi et al. Improving software defect prediction using hybrid machine learning models. *Journal of King Saud University – Computer and Information Sciences*, 35(8):101628, 2023. doi: 10.1016/j.jksuci.2023.101628.
- [18] PROMISE Repository. Software engineering dataset archive, 2025. URL <https://promise.site.uottawa.ca/SERepository/>.
- [19] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 2014 International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440, 2014. doi: 10.1145/2610384.2628055.
- [20] Martin Shepperd, David Bowes, and Tracy Hall. Researcher bias: The use of machine learning in software defect prediction. *IEEE Transactions on Software Engineering*, 40(6):603–616, 2014. doi: 10.1109/TSE.2014.2322358.
- [21] Martin Shepperd, Qinbao Song, Zhongbin Sun, and Carol Mair. Data quality: Some comments on the NASA software defect datasets. *IEEE Transactions on Software Engineering*, 39(9):1208–1215, 2013. doi: 10.1109/TSE.2013.11.
- [22] Leo Breiman. Random forests. *Machine Learning*, 45(1):5–32, 2001. doi: 10.1023/A:1010933404324.

- [23] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. In *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, pages 785–794, 2016. doi: 10.1145/2939672.2939785.
- [24] Biao Xu, Yutian Huang, et al. Predicting defects with code metrics using ensemble learning. In *Proceedings of the 27th IEEE International Symposium on Software Reliability Engineering (ISSRE)*, pages 26–37, 2016. doi: 10.1109/ISSRE.2016.22.
- [25] Taskeen Tabassum et al. Automated bug prediction using modern machine learning pipelines. *Journal of Software: Evolution and Process*, 35(3):e2478, 2023. doi: 10.1002/smr.2549.
- [26] Rafaqut Kazmi, Dayang N. A. Jawawi, Radziah Mohamad, and Imran Ghani. Performance comparison of multi-objective algorithms for test case prioritization during web application testing. *IEEE Access*, 10:31246–31258, 2022. doi: 10.1109/ACCESS.2022.3160163.
- [27] Rafaqut Kazmi, Dayang N. A. Jawawi, Radziah Mohamad, and Imran Ghani. Search for prioritized test cases in a multi-objective environment during web application testing. *Expert Systems with Applications*, 176:114843, 2021. doi: 10.1016/j.eswa.2021.114843.
- [28] Rafaqut Kazmi, Dayang N. A. Jawawi, Radziah Mohamad, and Imran Ghani. Search for prioritized test cases during web application testing. *Soft Computing*, 24(18): 14133–14151, 2020. doi: 10.1007/s00500-020-04787-2.
- [29] Liang Zhao, Xi Wu, and Yuankai Fang. A feature selection method for software defect prediction based on mutual information and ensemble learning. *IEEE Access*, 9: 97579–97591, 2021. doi: 10.1109/ACCESS.2021.3094576.